

Assignment 1

Magistrello Camilla (4512554)

1 Introduction

The aim of this assignment is to investigate the community structure of a network using different community detection algorithms.

The analyzed network represents similarities between anime extracted from the MyAnimeList dataset. Nodes represent anime, while edges represent similarity relationships based on shared genres. The dataset was filtered by considering only popular anime with at least 50,000 members and valid genre information.

Three community detection algorithms were applied: Louvain, Leiden, and Girvan–Newman. The obtained partitions were then compared using Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI) to evaluate the consistency between the detected communities.

2 Network Description

The graph was constructed from the filtered anime dataset. Each node represents an anime identified by its unique identifier. An undirected edge is created between two anime when they share at least two common genres.

The resulting network contains:

Property	Value
Number of nodes	3579
Number of edges	780393
Graph type	Undirected
Edge criterion	At least two shared genres

Table 1: Main properties of the anime similarity network.

Figure 1 shows the visualization of the anime similarity network.

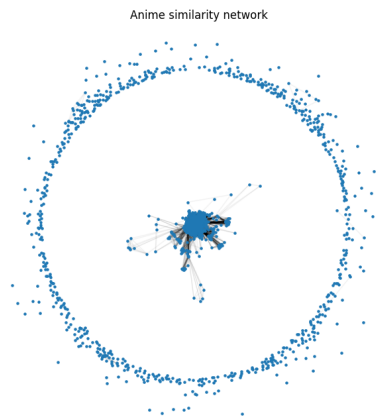


Figure 1: Visualization of the anime similarity network.

3 Community Detection

3.1 Louvain Algorithm

The Louvain algorithm was applied to the complete network. It detects communities by optimizing the modularity of the network, searching for a partition in which nodes have more connections within their community than expected.

The algorithm was executed using the Louvain implementation with `random_state=42`. The resulting partition assigns one community label to each node. The number of communities was then obtained by counting the distinct community labels.

The execution time was measured by recording the time immediately before and after the algorithm using Python’s `time` module.

The algorithm detected 733 communities with an execution time of 7.480 seconds.

3.2 Leiden Algorithm

The Leiden algorithm was applied to the complete graph using the `ModularityVertexPartition` implementation.

A fixed random number generator initialized with seed 42 was used to make the stochastic procedure reproducible. The NetworkX graph was first converted into an igraph graph. Leiden then assigned each node to a community. The number of communities was obtained by counting the communities in the resulting partition.

The execution time was measured using Python’s `time` module.

The algorithm detected 734 communities with an execution time of 0.546 seconds.

3.3 Girvan–Newman Algorithm

The Girvan–Newman algorithm detects communities by progressively removing edges with the highest edge betweenness centrality. Due to its computational complexity, the algorithm was applied to a reduced subgraph composed of 200 randomly selected nodes. The nodes of the reduced graph were selected using a fixed random seed (`seed=42`), ensuring that the same subgraph can be reproduced.

Girvan–Newman generates a sequence of increasingly fine community partitions. For each partition generated by the algorithm, the modularity was computed using the standard modularity measure. The partition with the highest modularity was retained as the final partition.

In this experiment, the partition with the maximum modularity contained 52 communities and achieved a modularity of $Q = 0.3391$. The complete procedure, including the generation of the partitions and the modularity evaluation, required 74.708 seconds.

3.4 Comparison

Algorithm	Communities	Execution Time (s)
Louvain	733	7.480
Leiden	734	0.546
Girvan–Newman	52	74.708

Table 2: Comparison of community detection algorithms.

Louvain and Leiden produced the same number of communities on the complete graph but Leiden required significantly less execution time, indicating better computational efficiency in this experiment.

Girvan–Newman required more execution time despite being applied to only 200 nodes, reflecting its substantially higher computational cost. However, execution times should not be directly compared because the algorithm was run on a different graph size. Therefore, its number of detected communities cannot be directly compared with those obtained by Louvain and Leiden on the complete graph.

4 Community Size Analysis

The size of each community was calculated by counting the number of nodes assigned to each community label. For each algorithm, the community labels were stored for all nodes and counted using Python’s `Counter` function.

The community sizes were then sorted in decreasing order to identify the ten largest communities. The community identifier refers to the original community label assigned by the corresponding algorithm.

Community	Size
0	1002
1	865
3	671
5	272
281	21
297	8
26	7
300	4
51	3
392	2

Table 3: Louvain

Community	Size
0	999
1	949
2	663
3	191
4	20
5	9
6	8
7	7
8	4
9	3

Table 4: Leiden

Community	Size
2	82
4	30
6	29
1	7
10	3
0	2
15	2
3	1
7	1
8	1

Table 5: Girvan–Newman

The community sizes show a highly uneven distribution, with a few large communities and many very small ones. Louvain and Leiden have very similar mean community sizes (4.883 and 4.876 nodes, respectively), while Girvan–Newman has a mean community size of approximately 3.846 nodes on the reduced 200-node graph.

5 Evaluation of Community Quality

Since no ground truth community assignment is available, the detected partitions were compared using Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI). These metrics measure the similarity between different community assignments without determining which partition is the correct one.

The comparison was performed between Louvain and Leiden, since both algorithms were applied to the complete graph.

Comparison	NMI	ARI
Louvain vs Leiden	0.9392	0.8751

Table 6: Similarity between Louvain and Leiden community partitions.

The obtained values (NMI = 0.9392 and ARI = 0.8751) indicate a strong agreement between the Louvain and Leiden partitions. Although the detected partitions are not identical, both algorithms identify a highly similar underlying community structure. Girvan–Newman was not included in this comparison because it was applied only to a reduced graph, whereas Louvain and Leiden were applied to the complete network.

6 Community Size Distribution

The distribution of community sizes was analyzed separately for the Louvain and Leiden partitions. The distributions are shown on logarithmic scales to better visualize the large range of community sizes and the frequency of small communities.

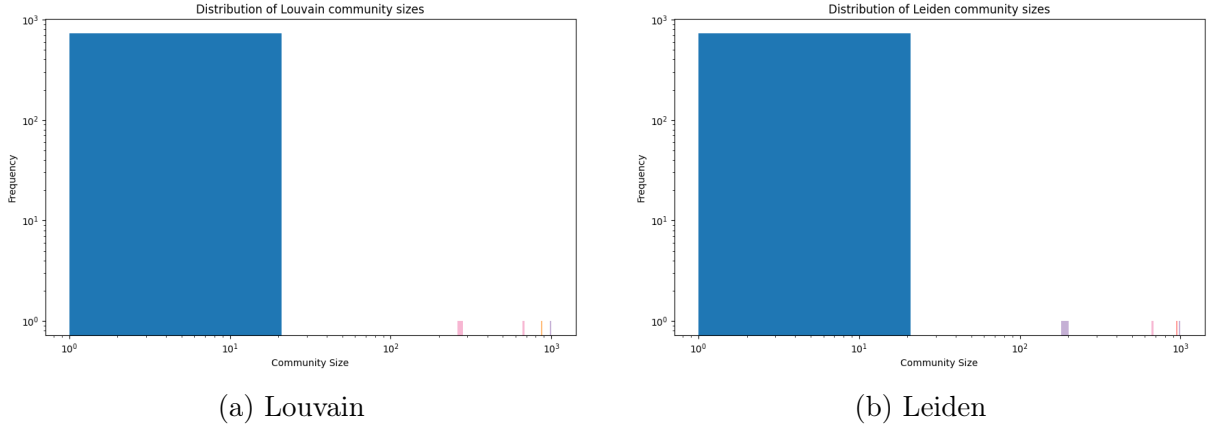


Figure 2: Distribution of community sizes for Louvain and Leiden.

Both distributions show a strongly uneven community structure, with a small number of very large communities and a large number of small communities. The four largest Louvain communities contain 1002, 865, 671, and 272 nodes, while the four largest Leiden communities contain 999, 949, 663, and 191 nodes.

7 Community Visualization

The detected communities were visualized by assigning different colors to the nodes according to their community label.

The following figure shows the community structures detected by the three algorithms. Louvain and Leiden were applied to the complete network, while Girvan–Newman was applied to the reduced 200-node subgraph.

For Louvain and Leiden, all detected communities are represented in the visualizations. Since hundreds of communities are present, individual colors cannot all be uniquely distinguished by eye.

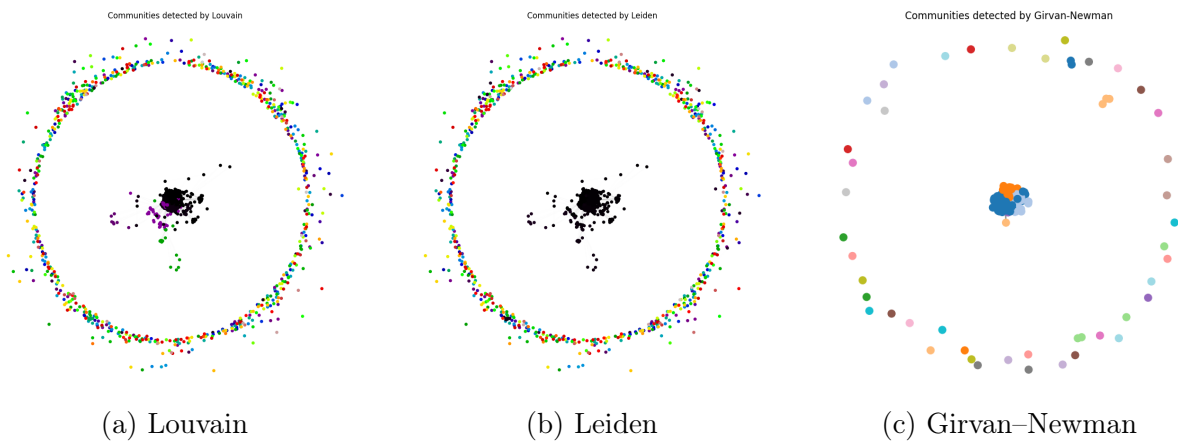


Figure 3: Community structures detected by the three algorithms.

8 Conclusions

This work investigated the community structure of the anime similarity network using Louvain, Leiden, and Girvan–Newman community detection algorithms.

The results show that Louvain and Leiden identified a highly consistent community structure, as indicated by the strong agreement measured by NMI and ARI. Their community size distributions were highly uneven, with a few large communities and many small ones. Girvan–Newman provided an additional perspective on community detection, but its application to a reduced subgraph limits the direct comparison with the other two algorithms.

Overall, Louvain and Leiden produced comparable community structures, while Leiden offered greater computational efficiency on the complete network.